

Foundations

Arabic text on LED dot-matrix displays

How the hardware works, from the wire up. No prior knowledge of registers, SPI or embedded toolchains assumed.



Three pieces

You type on a phone. Letters appear on a sign. Nothing is installed, and no router or internet is involved.

PHONE

Opens a web page. That is the entire app.

ESP32

A few-dollar computer with WiFi built in.

LED MATRIX

64 red LEDs in an 8×8 grid, plus a driver chip.



Five wires. That's all.



What each wire does

WIRE	NAME	JOB
VCC	Voltage	Power in — 5 volts to run the LEDs
GND	Ground	The return path. Every circuit needs one
DIN	Data In	The information itself, one bit at a time
CLK	Clock	A metronome. Says now — read the bit
CS	Chip Select	"I'm talking to you" / "message finished"



Why a clock wire at all?

The data wire only carries a voltage — high or low. Send ten 1 bits and the wire simply sits high the whole time.

The receiver cannot tell whether that is three bits or ten. **Nothing in the signal marks where one bit ends.**

The clock solves it. Every tick, the receiver samples the data line once. Ten ticks, ten bits.

Like spelling a word aloud over a bad phone line — the letters are the data, saying "next... next... next..." is the clock.



Ground is not optional

Voltage is always a **difference** between two points.

Saying "the data wire is at 3.3 V" means nothing unless both chips agree what **zero** is. The ground wire is that agreement.

Without it the display behaves randomly rather than failing cleanly — which makes it one of the more confusing faults to diagnose.



Why 5 V, not 3.3 V

The ESP32 runs internally at 3.3 V. The LED module wants 5 V.

Red LEDs need roughly 1.8–2 V before they conduct at all, and the driver needs headroom above that. At 3.3 V the LEDs are dim — and the driver's idea of "this counts as a logic 1" may stop matching what the ESP32 outputs.

You get flicker and garbage rather than an honest failure.

The board's VIN pin passes through the 5 V arriving over USB, which is exactly what the module wants.



One pin choice that matters

CS is on **GPIO27**, even though **GPIO12** sits physically closer.

GPIO12 is a **strapping pin**. At power-up the ESP32 reads it to choose its internal flash voltage, and it must read LOW at that instant.

A wire holding it high at boot makes the chip pick the wrong flash voltage and **refuse to start**. The board looks dead, and nothing about the symptom points at wiring.

Worth internalising early: a few pins have special jobs during boot. Check the datasheet before choosing pins, not after.

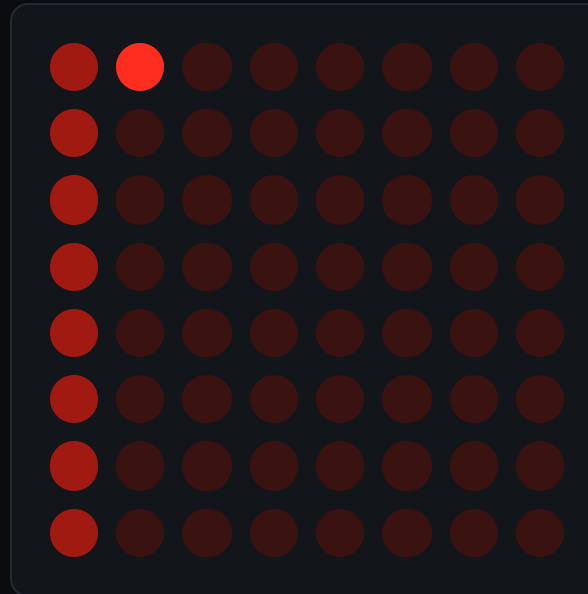


64 LEDs, 16 pins

Controlling each LED individually would need 64 wires. The module has 16.

Every LED sits where one **row** wire crosses one **column** wire. $8 + 8 = 16$
wires control 64 LEDs.

An LED lights only when its row is driven high and its column pulled low.



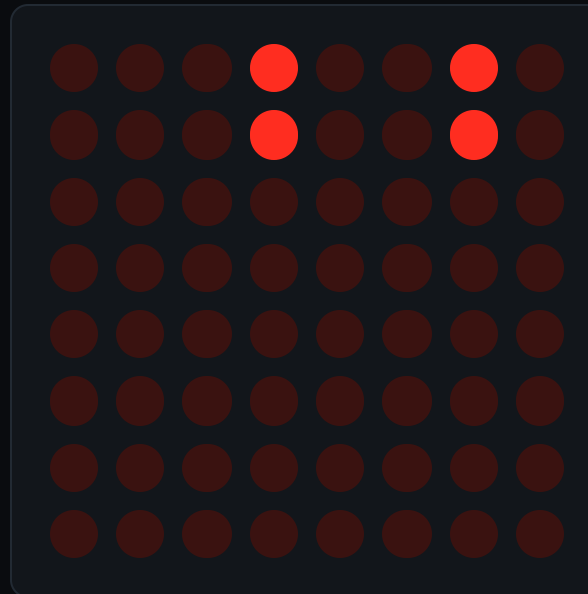
But there's a catch

62 THE MATRIX

FOUNDATIONS

Set rows 0 and 1 high, pull columns 3 and 6 low — and you light **four** LEDs, not two.

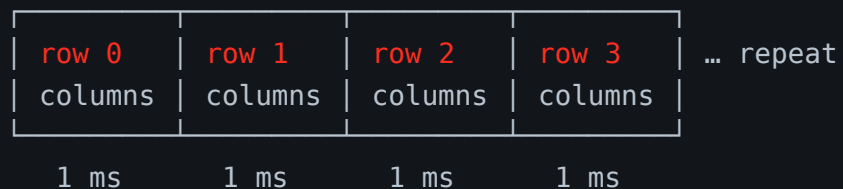
Rows and columns cannot be aimed **independently**. The grid always lights every intersection.



Multiplexing

The fix: never light more than one row at a time.

time →



Light row 0 with its pattern. Off. Row 1 with **its** pattern. Through all eight, then start again.



Persistence of vision

Each row is lit only $\frac{1}{8}$ of the time. But repeat the cycle faster than about 50 times a second and your eye cannot follow it.

The flashes blend into a steady image. **The same effect that makes film look like motion** rather than a sequence of stills.

The cost: every LED is off $\frac{7}{8}$ of the time, so the panel is dimmer than if all LEDs ran continuously. Drivers compensate by pushing more current during each brief on-period.



What a register is

A register is a small piece of memory **inside a chip** — not the microcontroller's RAM, but memory built into the peripheral itself.

bit 7 6 5 4 3 2 1 0

1	0	1	1	0	0	1	0
---	---	---	---	---	---	---	---

A row of switches that **stay where you put them**. In an LED driver, each bit is one LED.



Why chips use registers

A chip must be told what to do — but a pin for every setting would make it enormous. So it exposes a few registers and one small serial connection.

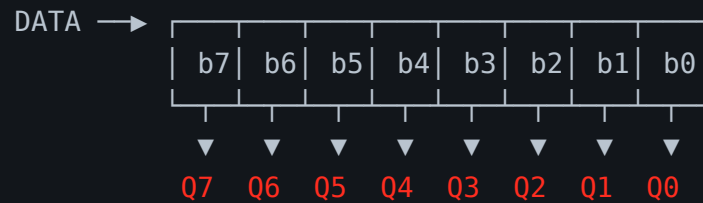
Configuration becomes **"write this number to that register"** instead of "add another wire".

ADDRESS	JOB
1	The 8 LEDs of row 0
2	The 8 LEDs of row 1
10	Brightness
12	On / off



The famous one: 74HC595

A **shift register** takes bits in one at a time and makes them available all at once.



Each clock tick pushes a bit in and shifts everything along — like people shuffling down a bench. After 8 ticks a **latch** pulse copies them to the pins.



Why the latch matters

Without it, the outputs would flicker through garbage while the bits shuffle past.

The latch means the outputs change **only when you say the data is complete.**

Why this is useful: three pins — data, clock, latch — become eight outputs. Chain two chips and three pins become sixteen.



What the 595 makes you add

The 74HC595 is general-purpose. It knows nothing about LEDs.

YOU MUST ADD	WHY
A resistor per LED	The 595 doesn't limit current — without them the LEDs burn out
A second chip	One 595 drives 8 columns; rows need their own driver
Transistors	A whole row draws more current than the chip can source
Multiplexing code	Your program must cycle the rows, fast, forever
Brightness logic	Any dimming, you write yourself



MAX7219: the same idea, specialised

74HC595

MAX7219

shift register	→	shift register
you add resistors	→	constant-current drivers built in
you add a row driver	→	row driver built in
you write multiplexing	→	multiplexing in hardware
you write dimming	→	16 brightness levels
you track the state	→	8 registers hold the image

One resistor sets the current for all 64 LEDs. **Not 64 resistors — one.**



The one that actually matters

The MAX7219 cycles the rows **itself**, continuously, forever.

Your program is free the instant the data is sent. **The entire refresh burden disappears.**

With a 595, that multiplexing loop must run non-stop. A few milliseconds of delay anywhere else in your program shows up as visible flicker — your code is permanently responsible for the display, on top of whatever else it is meant to do.



Which should you use?

	74HC595	MAX7219
Purpose	General shift register	LED matrix driver
Resistors	One per LED	One, total
Multiplexing	Your code, forever	Hardware
Brightness	Your circuit	Built in, 16 levels
Remembers image	No	Yes
Chips for one 8×8	2+ and transistors	1

The 595 is the better **teaching** chip — it makes you do the work. The MAX7219 is the better **building** chip — you don't have to.



SPI – three signals



Serial means bits travel one after another on a single wire, rather than eight wires side by side.



The MAX7219 has no way to answer

The **master** generates the clock and starts every exchange. Slaves only respond.

The MAX7219 has **no output wire at all**. It can never volunteer anything.

That has a real consequence: you cannot ask the module how it is wired internally. There is no path for an answer.

Anything you need to know about it has to be found by trying something and looking at the panel.



Sixteen bits per message

4 bits ignored	4 bits register address	8 bits data
-------------------	-------------------------------	----------------

Send 0x01, 0xFF — **register 1, all eight bits on** — and the top row lights fully.

The 595 has no address field at all. The MAX7219 spends 4 bits on one so a single chip can hold several registers. That is the difference between a generic part and a specialised one, visible right in the wire format.



Chaining surprises everyone

Bits shifted out of chip 1 flow into chip 2.

The word you send **first** ends up in the **last** chip of the chain.

Everything you send travels through the earlier chips to reach the later ones, pushing previous data along ahead of it.

So when writing code for a chain, you send data in reverse order.



How can an ESP32 serve a web page?

A web server is just a **program that waits for a request and sends text back**. That is the whole job.

```
browser → GET / HTTP/1.1  
server  → <!doctype html> ...
```

Nothing about that requires a big computer. **A few hundred lines, and the ESP32 has room to spare.**



The layers

Your HTML page (stored in flash)	← you write this
HTTP server – understands GET/POST	← library
TCP/IP stack – addresses, packets	← library
WiFi driver – radio, 802.11	← chip + library
WiFi radio hardware	← silicon

You write the top layer. Everything beneath ships with the SDK — that is what "batteries included" means.



Access Point mode

STATION

Joins an existing network. What your phone normally does.

ACCESS POINT

Becomes a network others join. What a router does.

The ESP32 runs as an access point, so the phone joins it directly. **No router, no internet, no SIM.**

A small DHCP server hands the phone an address — the same service your home router runs — and points it at the sign on 192.168.4.1.



Arduino vs PlatformIO vs ESP-IDF

These three are constantly confused, because they are not the same **kind** of thing.

ARDUINO

A **framework** — simplified functions, `setup()` and `loop()`.

ESP-IDF

A **framework** — Espressif's official SDK, the one Arduino-for-ESP32 is built on.

PLATFORMIO

A **build tool** — not a framework. It builds **either** of the other two.

"Arduino vs ESP-IDF" is a real choice. **"PlatformIO vs ESP-IDF" is a category error.**



Why this project uses ESP-IDF

REQUIREMENT	WHY ESP-IDF
AP + DHCP + HTTP server	All included, properly integrated
Scroll while serving pages	FreeRTOS tasks make it straightforward
Change pins without editing code	menuconfig
Precise SPI and timing control	Direct access to the drivers

With Arduino most of this is possible but fought for. With ESP-IDF it is what the SDK is **for**.



Installing and running it

```
# download the SDK – --recursive matters
git clone -b v5.3.2 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf && ./install.sh esp32

# activate – needed in EVERY new terminal
. ./export.sh

# the everyday command
idf.py -p /dev/ttyUSB0 flash monitor
```

Without --recursive you get an incomplete SDK and a confusing build failure. **Forgetting export.sh** gives `idf.py: command not found` — that is all it ever means.



How the code is organised

INCLUDES

Bringing in code someone else wrote. <...> is a system library, "..." is ours.

FUNCTIONS

Named blocks of work. What it returns, what it is called, what it takes.

MODULES

A .h + .c pair. The menu, and the kitchen.

Split them so you can change **how** something works without anything else caring — as long as the menu stays the same.



One job each

MODULE	RESPONSIBILITY
main	Startup and coordination
max7219	Talking to the LED driver chip
framebuffer	Storing the picture in memory
wifi_ap	Creating the WiFi network
http_ui	Serving the web page
font5x7	Drawing Latin letters, for testing

`max7219.c` knows about the chip but nothing about WiFi. `wifi_ap.c` knows about networks but nothing about LEDs. That is deliberate — you can understand one file without holding the whole project in your head.



Where the program starts

```
void app_main(void)
{
    max7219_init(...);    // 1. wake the display
    fb_init(...);         // 2. room for a picture
    wifi_ap_start(...);   // 3. create the network
    http_ui_start(...);   // 4. serve the page
}
```

Notice it does not loop forever. Under **FreeRTOS**, work happens in independent **tasks** the OS switches between.

That is exactly what would be painful in Arduino's single `loop()` — and the main reason this project uses ESP-IDF.



The whole chain

1. You type Arabic on your phone
2. The page turns letters into a grid of on/off dots
3. Dots are packed into bytes – 8 dots per byte
4. Sent over WiFi to the ESP32
5. Stored in its framebuffer
6. Sent over SPI to the MAX7219 registers
7. The chip multiplexes the rows continuously
8. Your eye blends the flashes into steady letters

Every step above has now been explained: what a register is, how the matrix multiplexes, which protocol carries the bits, why the ESP32 can host a page, and how the code is arranged to do it.

